جزوه درجه یک سیستم عامل

فصل ۴ - بخش ۶: ملاحظات نخ‌سازی (چالش‌های پیشرفته)

تا الان یاد گرفتیم که چطور نخ بسازیم و مدیریت کنیم. اما دنیای چندنخی پر از جزئیات و چالش‌های پنهان است. این بخش به ۵ مورد از مهم‌ترین "ملاحظات" می‌پردازد که هر برنامه‌نویس باید بداند.

۴-۶-۱: فراخوانی‌های سیستمی `fork()` و `exec()`

این دو فراخوانی (که از فصل ۳ به یاد داریم) در دنیای چندنخی رفتار عجیبی پیدا می‌کنند.

چالش `fork()`:

- **سوال:** وقتی یک *نخ* (از بین مثلاً ۵۰ نخ) تابع `fork()` را صدا می‌زند، پردازه فرزند جدید باید چگونه باشد؟
  1. **یک کپی کامل:** آیا پردازه فرزند هم باید ۵۰ نخ داشته باشد (کپی کامل والد)؟
  2. **فقط نخِ فراخوانی‌کننده:** آیا پردازه فرزند باید فقط یک نخ داشته باشد (فقط نخی که `fork` را صدا زد)؟
- **پاسخ:** اکثر سیستم‌های یونیکس **هر دو نسخه** را ارائه می‌دهند! اینکه کدام بهتر است، به کاری که می‌خواهید بکنید بستگی دارد:
  - **اگر بلافاصله `exec` می‌کنید:** منطقی است که از نسخه "تک نخی" استفاده کنید. چرا ۵۰ نخ را کپی کنیم، وقتی قرار است *بلافاصله* با `exec` همه‌چیز را پاک کنیم و یک برنامه جدید اجرا کنیم؟
  - **اگر `exec` نمی‌کنید:** اگر پردازه فرزند قرار است به کار والد ادامه دهد، باید تمام نخ‌ها را کپی کند.

رفتار `exec()`:

- `exec` ساده و بی‌رحم است.
- اگر *هر نخی* `exec` را صدا بزند، کل پردازه (شامل *تمام* نخ‌های دیگر، چه در حال اجرا باشند چه نباشند) به طور کامل **جایگزین** می‌شود. `exec` کل "کارگاه" را با یک کارگاه جدید عوض می‌کند.

۴-۶-۲: رسیدگی به سیگنال (Signal Handling)

سیگنال‌ها (که در یونیکس رایج‌اند) روشی برای اطلاع‌رسانی به پردازه در مورد یک "رویداد" هستند (مثلاً `Ctrl+C`، خطای تقسیم بر صفر، ...).

مشکل در دنیای چندنخی: در یک پردازه تک-نخی، معلوم بود سیگنال باید به کی تحویل داده شود (به همان پردازه). اما حالا که پردازه ما ۵۰ نخ دارد، سیگنال باید به *کدام نخ* تحویل داده شود؟

انواع سیگنال‌ها:

1. **سیگنال همگام (Synchronous):**
   - علتش، خودِ نخ است (مثلاً خطای تقسیم بر صفر یا دسترسی به حافظه غیرمجاز).
   - **راه‌حل:** ساده است. سیگنال باید *دقیقاً* به همان نخی تحویل داده شود که خطا را ایجاد کرده.
1. **سیگنال ناهمگام (Asynchronous):**
   - علتش، یک رویداد خارجی است (مثلاً کاربر `Ctrl+C` را زده).

- **راه‌حل (گزینه‌های ممکن):**
  - الف) سیگنال به *تمام* نخ‌ها تحویل داده شود.
  - ب) سیگنال به *یک نخ خاص* (مثلاً نخ اصلی) تحویل داده شود.
  - ج) سیگنال به *اولین* نخی که آن را "مسدود" (Block) نکرده، تحویل داده شود (این روش رایج است).
- **نکته:** مدیریت سیگنال در برنامه‌های چندنخی بسیار پیچیده است. به همین دلیل برخی سیستم‌ها (مثل ویندوز) به جای سیگنال از مکانیسم تمیزتری به نام **APC (فراخوانی ناهمگام روال)** استفاده می‌کنند که مستقیماً به یک *نخ خاص* تحویل داده می‌شود.

۴-۶-۳: برکناری نخ (Thread Cancellation)

گاهی لازم است یک نخ را قبل از اینکه کارش تمام شود، "بکُشیم" (Cancel کنیم).

- **مثال:** در مرورگر وب، وقتی دکمه Stop را می‌زنید، باید تمام نخ‌هایی که در حال دانلود عکس‌ها هستند، بلافاصله متوقف (Cancel) شوند.
- به نخی که قرار است کشته شود، **"نخ هدف" (Target Thread)** می‌گویند.

دو روش برای برکناری وجود دارد:

1. **برکناری ناهمگام (Asynchronous Cancellation):** (روش خطرناک)
   - **ایده:** یک نخ، نخ هدف را *بلافاصله* و بدون اخطار قبلی می‌کُشد.
   - **مشکل:** بسیار خطرناک است! مثل این است که کارگر را وسط کار (مثلاً وقتی یک فایل مهم را باز کرده یا حافظه‌ای را قفل کرده) از کارگاه بیرون بیندازیم. منابع سیستم ممکن است قفل بمانند و آزاد نشوند.
1. **برکناری تأخیردار (Deferred Cancellation):** (روش امن و پیش‌فرض)
   - **ایده:** یک نخ فقط "درخواست" برکناری را برای نخ هدف می‌فرستد (یک "پرچم برکناری" را `true` می‌کند).
   - نخ هدف، خودش به صورت دوره‌ای (مثلاً هر چند ثانیه) آن پرچم را چک می‌کند (`pthread_testcancel()`).
   - اگر دید که پرچم `true` شده، می‌فهمد که باید بمیرد. پس کارهایش را تمیز جمع و جور می‌کند (منابع را آزاد می‌کند) و به صورت "داوطلبانه" خاتمه می‌یابد.
   - به نقاطی که نخ پرچمش را چک می‌کند، **"نقاط امن" (Cancellation Points)** گفته می‌شود. (اکثر توابع مسدودشونده مثل `read` به صورت خودکار یک نقطه برکناری هستند).
- **در جاوا:** برکناری با متد `interrupt()` انجام می‌شود که دقیقاً مدل "تأخیردار" است. این متد فقط یک پرچم را `true` می‌کند و خود نخ هدف باید با `isInterrupted()` آن را چک کند.

۴-۶-۴: حافظه محلی نخ (Thread-Local Storage - TLS)

- **مشکل:** ما یاد گرفتیم که نخ‌ها در داده‌های پردازه (متغیرهای سراسری) **مشترک** هستند. اما گاهی اوقات ما نیاز داریم که یک متغیر، *ظاهراً* سراسری باشد، اما هر نخ کپی *شخصی* خودش را از آن داشته باشد.
- **مثال:** فرض کنید در یک سرور وب، هر نخ یک "شماره تراکنش" (Transaction ID) منحصر به فرد دارد. ما می‌خواهیم این ID در تمام توابع آن نخ در دسترس باشد، اما با ID نخ‌های دیگر قاطی نشود.
- **راه‌حل: TLS (حافظه محلی نخ).**
  - TLS یک حافظه "نیمه-سراسری" ایجاد می‌کند.

- این داده‌ها `static` (سراسر عمر نخ) هستند، اما `local` (مختص همان نخ).
- هر نخ فقط به کپی خودش دسترسی دارد.

۴-۶-۵: فعال‌سازی زمانبند (Scheduler Activations)

این یک مفهوم پیشرفته برای هماهنگی بین نخ‌های سطح کاربر و نخ‌های سطح هسته در مدل‌های "چند-به-چند" است.

- **واسطه (LWP):** سیستم یک واسطه به نام **"پردازه سبک‌وزن" (LWP - Lightweight Process)** بین نخ کاربر و نخ هسته قرار می‌دهد.
  - از دید نخ کاربر، LWP مثل یک CPU مجازی است.
  - از دید هسته، LWP همان چیزی است که به نخ هسته متصل می‌شود.
- **مشکل:** اگر نخ کاربر یک فراخوانی مسدودشونده انجام دهد، LWP و نخ هسته متصل به آن هم مسدود می‌شوند. حالا کتابخانه سطح کاربر از کجا بفهمد که باید یک نخ کاربر دیگر را روی LWP دیگری بفرستد؟
- **راه‌حل (Upcalls):**

1. وقتی نخ هسته مسدود می‌شود، هسته یک **"ندا" (Upcall)** به کتابخانه نخ (در سطح کاربر) می‌فرستد و می‌گوید: "هی، LWP شماره ۵ مسدود شد!"
2. کتابخانه (که به این ندا گوش می‌دهد) متوجه می‌شود و بلافاصله یک نخ کاربر دیگر را که آماده بود، روی یک LWP آزاد دیگر زمان‌بندی می‌کند.
3. وقتی کار نخ مسدود شده تمام شد، هسته دوباره یک "ندا" (Upcall) می‌فرستد که: "LWP شماره ۵ آزاد شد."

این مکانیسم "ندا" (Upcall) به کتابخانه نخ اجازه می‌دهد تا به صورت هوشمندانه و کارآمد، نخ‌های کاربر را روی LWPهای موجود مدیریت کند.